D7041-LAB 3: Reservoir Computing

```
"""
D7041-Lab 2
Deborah Aittoklllio debait-2
Joel Willén Joewil-2

"""
```

Imports

```
import numpy as np  # For core numerical computations
import matplotlib.pyplot as plt  # For plotting results
from sklearn.metrics import confusion_matrix # For evaluating
classification performance (Used in Section 4)
import seaborn as sns
from tqdm import tqdm
np.random.seed(42)  # For reproducibility

# Make plots look nicer
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (12, 6)
```

Section 2: Echo State Networks (Implementation)

```
"""

ANSWERS TO GUIDING QUESTIONS:

1. What does teacher forcing mean?
Teacher forcing means that during training, instead of feeding the
network's own
predictions back as input, we feed the actual correct values (ground
truth).
This helps the network learn more stable patterns because it doesn't
get confused
by its own errors during training.

2. What is ridge regression? Why is it useful?
Ridge regression is like normal linear regression but with an extra
penalty term
that prevents the weights from becoming too large. It's useful because
it stops
overfitting and makes the network more stable, especially when we have
feedback
connections. The penalty is controlled by a parameter beta.

3. How to achieve echo state property?
The echo state property means the network state depends only on recent
```

```
inputs,
not on where it started. We achieve this by keeping the spectral
radius (largest
eigenvalue) of the reservoir weight matrix W below 1 (or slightly
above with
strong inputs). This keeps the network stable.

4. What is Root-Mean-Square Error (RMSE)?
RMSE measures how far our predictions are from the true values on
average.
It's calculated as: sqrt(mean((predictions - actual_values)^2))
Lower RMSE means better predictions.

"""

class ESN:
    def __init__(self, input_size, reservoir_size, output_size,
                 spectral_radius=0.9, input_scaling=1.0,
random_seed=None):
        """
        Initialize the Echo State Network

        Parameters:
        - input_size: number of input dimensions
        - reservoir_size: number of neurons in reservoir
        - output_size: number of output dimensions
        - spectral_radius: scaling for reservoir weights (default 0.9)
        - input_scaling: scaling for input weights (default 1.0)
        - random_seed: for reproducibility
        """
        if random_seed is not None:
            np.random.seed(random_seed)

        self.input_size = input_size
        self.reservoir_size = reservoir_size
        self.output_size = output_size

        # Create input weights Win (including bias)
        # Shape: (reservoir_size, 1 + input_size)
        # The first column is for bias (always gets input 1)
        self.Win = np.random.uniform(-1, 1, (reservoir_size, 1 +
input_size))
        self.Win = self.Win * input_scaling

        # Create reservoir weights W
        # Start with random values
        self.W = np.random.uniform(-1, 1, (reservoir_size,
reservoir_size))

        # Scale W to have the desired spectral radius
```

```
        # First find current spectral radius
        eigenvalues = np.linalg.eigvals(self.W)
        current_spectral_radius = np.max(np.abs(eigenvalues))

        # Scale to desired spectral radius
        self.W = self.W * (spectral_radius / current_spectral_radius)

        # Output weights - will be trained later
        self.Wout = None

        # Keep track of current reservoir state
        self.state = np.zeros((reservoir_size, 1))

    def update_state(self, input_val):
        """
        Update the reservoir state with new input

        Parameters:
        - input_val: current input value (can be scalar or array)

        Returns:
        - new reservoir state
        """
        # Make sure input is the right shape
        if np.isscalar(input_val):
            input_val = np.array([[input_val]])
        elif input_val.ndim == 1:
            input_val = input_val.reshape(-1, 1)

        # Add bias (constant 1) to input
        input_with_bias = np.vstack([[[1]], input_val])

        # Update equation: x(n) = tanh(Win * [1; u(n)] + W * x(n-1))
        # Note: we're using alpha=1 (no leaky integration)
        self.state = np.tanh(self.Win @ input_with_bias + self.W @
self.state)

        return self.state

    def reset_state(self):
        """Reset reservoir state to zero"""
        self.state = np.zeros((self.reservoir_size, 1))

    def train(self, inputs, targets, washout=0, beta=1e-6):
        """
        Train the output weights using ridge regression

        Parameters:
        - inputs: training input sequence (T x input_size) or (T,) for
1D
```

```
        - targets: training target sequence (T x output_size) or (T,)
for 1D
        - washout: number of initial timesteps to discard
        - beta: ridge regression regularization parameter

        Returns:
        - training error (RMSE)
        """
        # Reset state before training
        self.reset_state()

        # Make sure inputs and targets are 2D
        if inputs.ndim == 1:
            inputs = inputs.reshape(-1, 1)
        if targets.ndim == 1:
            targets = targets.reshape(-1, 1)

        n_samples = inputs.shape[0]

        # Collect reservoir states
        states = np.zeros((self.reservoir_size, n_samples))

        for t in range(n_samples):
            states[:, t:t+1] = self.update_state(inputs[t])

        # Remove washout period
        states = states[:, washout:]
        targets_train = targets[washout:]
        inputs_train = inputs[washout:]

        # Create extended state matrix [1; u; x]
        # This includes bias, input, and reservoir state
        bias = np.ones((1, states.shape[1]))
        extended_states = np.vstack([bias, inputs_train.T, states])

        # Ridge regression to find Wout
        # Formula: Wout = Y * X^T * (X * X^T + beta*I)^-1
        # where Y is targets and X is extended_states

        # Calculate X * X^T
        XXt = extended_states @ extended_states.T

        # Add regularization term
        XXt_reg = XXt + beta * np.eye(extended_states.shape[0])

        # Solve for Wout
        self.Wout = targets_train.T @ extended_states.T @
np.linalg.inv(XXt_reg)

        # Calculate training error
```

```
        predictions = self.Wout @ extended_states
        error = np.sqrt(np.mean((predictions.T - targets_train)**2))

        return error

    def predict(self, inputs):
        """
        Make predictions on new data

        Parameters:
        - inputs: input sequence (T x input_size) or (T,)

        Returns:
        - predictions (T x output_size)
        """
        if self.Wout is None:
            raise Exception("Network not trained yet! Call train()
first.")

        # Make sure inputs are 2D
        if inputs.ndim == 1:
            inputs = inputs.reshape(-1, 1)

        n_samples = inputs.shape[0]
        predictions = np.zeros((n_samples, self.output_size))

        for t in range(n_samples):
            # Update state
            state = self.update_state(inputs[t])

            # Create extended state [1; u; x]
            bias = np.array([[1]])
            extended_state = np.vstack([bias, inputs[t].reshape(-1,
1), state])

            # Predict output
            predictions[t] = (self.Wout @ extended_state).flatten()

        return predictions

    def generate(self, n_steps, initial_input=None):
        """
        Generate predictions in generative mode (output feeds back as
input)

        Parameters:
        - n_steps: number of steps to generate
        - initial_input: starting input value

        Returns:
```

```
        - generated sequence (n_steps,)
        """
        if self.Wout is None:
            raise Exception("Network not trained yet!")

        predictions = np.zeros(n_steps)

        # Start with initial input or current state
        if initial_input is not None:
            current_input = initial_input
        else:
            current_input = 0.0

        for t in range(n_steps):
            # Update state with current input
            state = self.update_state(current_input)

            # Create extended state
            bias = np.array([[1]])
            input_reshaped = np.array([[current_input]])
            extended_state = np.vstack([bias, input_reshaped, state])

            # Generate prediction
            prediction = (self.Wout @ extended_state).flatten()[0]
            predictions[t] = prediction

            # Feed prediction back as next input
            current_input = prediction

        return predictions



# Helper function to calculate RMSE
def calculate_rmse(predictions, targets):
    """
    Calculate Root Mean Square Error

    Parameters:
    - predictions: predicted values
    - targets: true values

    Returns:
    - RMSE value
    """
    return np.sqrt(np.mean((predictions - targets)**2))


# Test the implementation with a simple example
print("ESN Implementation Complete!")
print("\nTesting with a simple sine wave...")
```

```
# Generate simple test data
test_steps = 100
test_data = np.sin(np.linspace(0, 4*np.pi, test_steps))

# Create small ESN for testing
test_esn = ESN(input_size=1, reservoir_size=50, output_size=1,
               spectral_radius=0.8, input_scaling=0.5)

# Train on first half
train_data = test_data[:50]
targets = test_data[1:51]  # One step ahead
inputs = test_data[:50]

error = test_esn.train(inputs, targets, washout=10, beta=1e-6)
print(f"Training RMSE: {error:.6f}")

print("\n✓ ESN implementation working!")
```

```
ESN Implementation Complete!

Testing with a simple sine wave...
Training RMSE: 0.000086

✓ ESN implementation working!
```

Section 3: Modeling of Dynamic Systems with ESNs

Section 3.1 Sinusodial Signal Detection

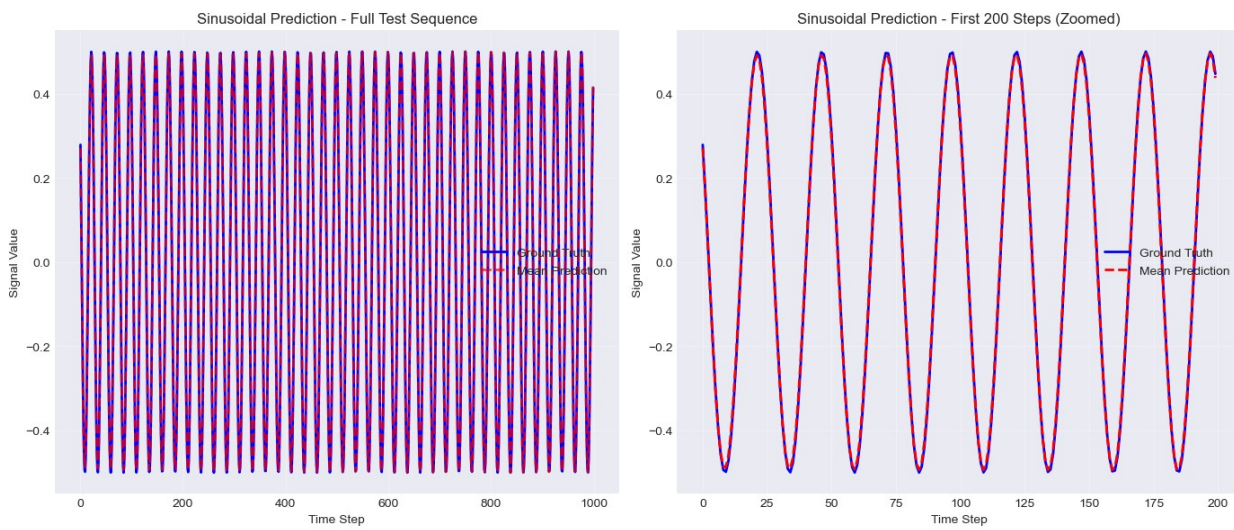
```
# Parameters from the assignment
n_timesteps = 4000
train_len = 3000
test_len = 1000
washout = 1000  # Discard first 1000 states during training

# ESN Configuration
reservoir_size = 1000
input_scaling = 0.2
spectral_radius = 0.8
beta = 1e-6  # Ridge regression parameter

# Number of trials
n_trials = 10

# Generate the sinusoidal signal
# y(n) = 0.5 * sin(n/4)
n = np.arange(1, n_timesteps + 1)
y_signal = 0.5 * np.sin(n / 4)

# Split into training and testing
```

```python
y_train = y_signal[:train_len]
y_test = y_signal[train_len:train_len + test_len]

# Storage for results from multiple trials
all_predictions = np.zeros((n_trials, test_len))
all_rmse = np.zeros(n_trials)

print("Running 10 trials of sinusoidal prediction...")

for trial in range(n_trials):
    print(f"Trial {trial + 1}/10...", end=" ")

    # Create new ESN with different random initialization each time
    esn = ESN(input_size=1,
              reservoir_size=reservoir_size,
              output_size=1,
              spectral_radius=spectral_radius,
              input_scaling=input_scaling,
              random_seed=trial)  # Different seed each trial

    # Training phase with teacher forcing
    # Input: y(n-1), Target: y(n)
    train_inputs = y_train[:-1]  # y(0) to y(2998)
    train_targets = y_train[1:]   # y(1) to y(2999)

    train_error = esn.train(train_inputs, train_targets,
                            washout=washout, beta=beta)

    # Testing phase - Generative mode
    # Start by feeding the last training value
    esn.reset_state()

    # Warm up the reservoir with training data
    for t in range(len(y_train)):
        esn.update_state(y_train[t])

    # Now generate predictions
    predictions = np.zeros(test_len)
    current_input = y_train[-1]  # Start with last training value

    for t in range(test_len):
        # Update state with current input
        state = esn.update_state(current_input)

        # Create extended state [1; u; x] for prediction
        bias = np.array([[1]])
        input_reshaped = np.array([[current_input]])
        extended_state = np.vstack([bias, input_reshaped, state])

        # Predict next value
```

```
        prediction = (esn.Wout @ extended_state).flatten()[0]
        predictions[t] = prediction

        # Feed prediction back as next input (generative mode)
        current_input = prediction

    # Calculate RMSE
    rmse = calculate_rmse(predictions, y_test)
    all_predictions[trial] = predictions
    all_rmse[trial] = rmse

    print(f"RMSE: {rmse:.6f}")

# Calculate mean predictions across all trials
mean_predictions = np.mean(all_predictions, axis=0)
mean_rmse = np.mean(all_rmse)
std_rmse = np.std(all_rmse)

print(f"\n{'='*50}")
print(f"Results Summary:")
print(f"Mean RMSE: {mean_rmse:.6f} ± {std_rmse:.6f}")
print(f"{'='*50}")

# Plotting
plt.figure(figsize=(14, 6))

# Plot 1: Full test sequence
plt.subplot(1, 2, 1)
plt.plot(y_test, 'b-', label='Ground Truth', linewidth=2)
plt.plot(mean_predictions, 'r--', label='Mean Prediction',
linewidth=2, alpha=0.7)
plt.xlabel('Time Step')
plt.ylabel('Signal Value')
plt.title('Sinusoidal Prediction - Full Test Sequence')
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 2: Zoomed in view of first 200 steps
plt.subplot(1, 2, 2)
zoom_steps = 200
plt.plot(y_test[:zoom_steps], 'b-', label='Ground Truth', linewidth=2)
plt.plot(mean_predictions[:zoom_steps], 'r--', label='Mean
Prediction', linewidth=2)
plt.xlabel('Time Step')
plt.ylabel('Signal Value')
plt.title('Sinusoidal Prediction - First 200 Steps (Zoomed)')
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

```
Running 10 trials of sinusoidal prediction...
Trial 1/10... RMSE: 0.004770
Trial 2/10... RMSE: 0.011071
Trial 3/10... RMSE: 0.020009
Trial 4/10... RMSE: 0.008251
Trial 5/10... RMSE: 0.005665
Trial 6/10... RMSE: 0.013677
Trial 7/10... RMSE: 0.011136
Trial 8/10... RMSE: 0.012778
Trial 9/10... RMSE: 0.003820
Trial 10/10... RMSE: 0.015209

==================================================
Results Summary:
Mean RMSE: 0.010639 ± 0.004840
==================================================
```



```python
import numpy as np  # For core numerical computations
import matplotlib.pyplot as plt  # For plotting results
from sklearn.metrics import confusion_matrix # For evaluating
classification performance (Used in Section 4)
import seaborn as sns
from tqdm import tqdm
np.random.seed(42)  # For reproducibility

# check for intuition at
http://users.fs.cvut.cz/ivo.bukovsky/PVVR/prace_studentu/Mackey_Glass_
time_series_prediction_Fuzzy_Lookup.pdf

def mackey_glass(tau, length):
    betta = 0.2
    gamma = 0.1
    a = 10
    delay = 100  # will cut these initial steps

    y = np.random.rand(tau)  # random initial values between 0 and 1

    # Euler's method to solve the differential equation
    for t in range(tau - 1, length + delay + tau - 1):
        current = (
            y[t]
            + (betta * y[t - (tau - 1)] / (1 + np.power(y[t - (tau -
1)], a)))
            - gamma * y[t]
        )
        y = np.append(y, current)

    y = y - 1      # get rid of mean
    y = np.tanh(y) # squeeze with tanh

    # return the last 'length' elements (cut off transients)
    return y[tau + delay:length + delay + tau]

class ESN:
    def __init__(self, input_size, reservoir_size, output_size,
                 spectral_radius=0.9, input_scaling=1.0,
random_seed=None):
        """
        Initialize the Echo State Network

        Parameters:
        - input_size: number of input dimensions
        - reservoir_size: number of neurons in reservoir
        - output_size: number of output dimensions
        - spectral_radius: scaling for reservoir weights (default 0.9)
        - input_scaling: scaling for input weights (default 1.0)
        - random_seed: for reproducibility
```

```
        """
        if random_seed is not None:
            np.random.seed(random_seed)

        self.input_size = input_size
        self.reservoir_size = reservoir_size
        self.output_size = output_size

        # Create input weights Win (including bias)
        # Shape: (reservoir_size, 1 + input_size)
        # The first column is for bias (always gets input 1)
        self.Win = np.random.uniform(-1, 1, (reservoir_size, 1 +
input_size))
        self.Win = self.Win * input_scaling

        # Create reservoir weights W
        # Start with random values
        self.W = np.random.uniform(-1, 1, (reservoir_size,
reservoir_size))

        # Scale W to have the desired spectral radius
        # First find current spectral radius
        eigenvalues = np.linalg.eigvals(self.W)
        current_spectral_radius = np.max(np.abs(eigenvalues))

        # Scale to desired spectral radius
        self.W = self.W * (spectral_radius / current_spectral_radius)

        # Output weights - will be trained later
        self.Wout = None

        # Keep track of current reservoir state
        self.state = np.zeros((reservoir_size, 1))

    def update_state(self, input_val):
        """
        Update the reservoir state with new input

        Parameters:
        - input_val: current input value (can be scalar or array)

        Returns:
        - new reservoir state
        """
        # Make sure input is the right shape
        if np.isscalar(input_val):
            input_val = np.array([[input_val]])
        elif input_val.ndim == 1:
            input_val = input_val.reshape(-1, 1)
```

```
        # Add bias (constant 1) to input
        input_with_bias = np.vstack([[1]], input_val])

        # Update equation: x(n) = tanh(Win * [1; u(n)] + W * x(n-1))
        # Note: we're using alpha=1 (no leaky integration)
        self.state = np.tanh(self.Win @ input_with_bias + self.W @
self.state)

        return self.state

    def reset_state(self):
        """Reset reservoir state to zero"""
        self.state = np.zeros((self.reservoir_size, 1))

    def train(self, inputs, targets, washout=0, beta=1e-6):
        """
        Train the output weights using ridge regression

        Parameters:
        - inputs: training input sequence (T x input_size) or (T,) for
1D
        - targets: training target sequence (T x output_size) or (T,)
for 1D
        - washout: number of initial timesteps to discard
        - beta: ridge regression regularization parameter

        Returns:
        - training error (RMSE)
        """
        # Reset state before training
        self.reset_state()

        # Make sure inputs and targets are 2D
        if inputs.ndim == 1:
            inputs = inputs.reshape(-1, 1)
        if targets.ndim == 1:
            targets = targets.reshape(-1, 1)

        n_samples = inputs.shape[0]

        # Collect reservoir states
        states = np.zeros((self.reservoir_size, n_samples))

        for t in range(n_samples):
            states[:, t:t+1] = self.update_state(inputs[t])

        # Remove washout period
        states = states[:, washout:]
        targets_train = targets[washout:]
```

```
        # Create extended state matrix [1; x]
        # Bias + reservoir state
        bias = np.ones((1, states.shape[1]))
        extended_states = np.vstack([bias, states])


        # Ridge regression to find Wout
        # Formula: Wout = Y * X^T * (X * X^T + beta*I)^-1
        # where Y is targets and X is extended_states

        # Calculate X * X^T
        XXt = extended_states @ extended_states.T

        # Add regularization term
        XXt_reg = XXt + beta * np.eye(extended_states.shape[0])

        # Solve for Wout
        self.Wout = targets_train.T @ extended_states.T @
np.linalg.inv(XXt_reg)

        # Calculate training error
        predictions = self.Wout @ extended_states
        error = np.sqrt(np.mean((predictions.T - targets_train)**2))

        return error

    def predict(self, inputs):
        """
        Make predictions on new data

        Parameters:
        - inputs: input sequence (T x input_size) or (T,)

        Returns:
        - predictions (T x output_size)
        """
        if self.Wout is None:
            raise Exception("Network not trained yet! Call train()
first.")

        # Make sure inputs are 2D
        if inputs.ndim == 1:
            inputs = inputs.reshape(-1, 1)

        n_samples = inputs.shape[0]
        predictions = np.zeros((n_samples, self.output_size))

        for t in range(n_samples):
            # Update state
            state = self.update_state(inputs[t])
```

```
            # Create extended state [1; x]
            bias = np.array([[1]])
            extended_state = np.vstack([bias, state])


            # Predict output
            predictions[t] = (self.Wout @ extended_state).flatten()

        return predictions

    def generate(self, n_steps, initial_input=None):
        """
        Generate predictions in generative mode (output feeds back as
input)

        Parameters:
        - n_steps: number of steps to generate
        - initial_input: starting input value

        Returns:
        - generated sequence (n_steps,)
        """
        if self.Wout is None:
            raise Exception("Network not trained yet!")

        predictions = np.zeros(n_steps)

        # Start with initial input or current state
        if initial_input is not None:
            current_input = initial_input
        else:
            current_input = 0.0

        for t in range(n_steps):
            # Update state with current input
            state = self.update_state(current_input)

            # Create extended state
            bias = np.array([[1]])
            extended_state = np.vstack([bias, state])


            # Generate prediction
            prediction = (self.Wout @ extended_state).flatten()[0]
            predictions[t] = prediction

            # Feed prediction back as next input
            current_input = prediction

        return predictions
```

```python
# Helper function to calculate RMSE
def calculate_rmse(predictions, targets):
    """
    Calculate Root Mean Square Error

    Parameters:
    - predictions: predicted values
    - targets: true values

    Returns:
    - RMSE value
    """
    return np.sqrt(np.mean((predictions - targets)**2))


# Test the implementation with a simple example
print("ESN Implementation Complete!")
print("\nTesting with a simple sine wave...")

# Generate simple test data
test_steps = 100
test_data = np.sin(np.linspace(0, 4*np.pi, test_steps))

# Create small ESN for testing
test_esn = ESN(input_size=1, reservoir_size=50, output_size=1,
               spectral_radius=0.8, input_scaling=0.5)

# Train on first half
train_data = test_data[:50]
targets = test_data[1:51]  # One step ahead
inputs = test_data[:50]

error = test_esn.train(inputs, targets, washout=10, beta=1e-6)
print(f"Training RMSE: {error:.6f}")

print("\n✓ ESN implementation working!")




# Time series Classification addons
# Task 1 & 2


def z_norm_rows(X, eps=1e-8):
    mu = X.mean(axis=1, keepdims=True)
    sd = X.std(axis=1, keepdims=True)
    return (X - mu) / (sd + eps)
```

```
train_file = "SwedishLeaf_TRAIN.tsv"
test_file  = "SwedishLeaf_TEST.tsv"

train_raw = np.loadtxt(train_file, delimiter='\t')
test_raw  = np.loadtxt(test_file, delimiter='\t')

# First column is label, rest are time-series values
y_train_labels = train_raw[:, 0].astype(int)
X_train_series = train_raw[:, 1:]

y_test_labels  = test_raw[:, 0].astype(int)
X_test_series  = test_raw[:, 1:]

X_train_series = z_norm_rows(X_train_series)
X_test_series  = z_norm_rows(X_test_series)

n_train, series_len = X_train_series.shape
n_test, _           = X_test_series.shape

classes = np.unique(y_train_labels)
n_classes = len(classes)

print(f"Train size: {n_train}, Test size: {n_test}, Series length:
{series_len}, Classes: {n_classes}")


# Task 7

def encode_sequence_last_state(esn, sequence_1d):
    esn.reset_state() #Reset Esn
    for val in sequence_1d:
        esn.update_state(val) #Feed sequence
    return esn.state.flatten()  # shape: (reservoir_size,). 1D array
```

```
ESN Implementation Complete!

Testing with a simple sine wave...
Training RMSE: 0.000070

✓ ESN implementation working!
Train size: 500, Test size: 625, Series length: 128, Classes: 15
```

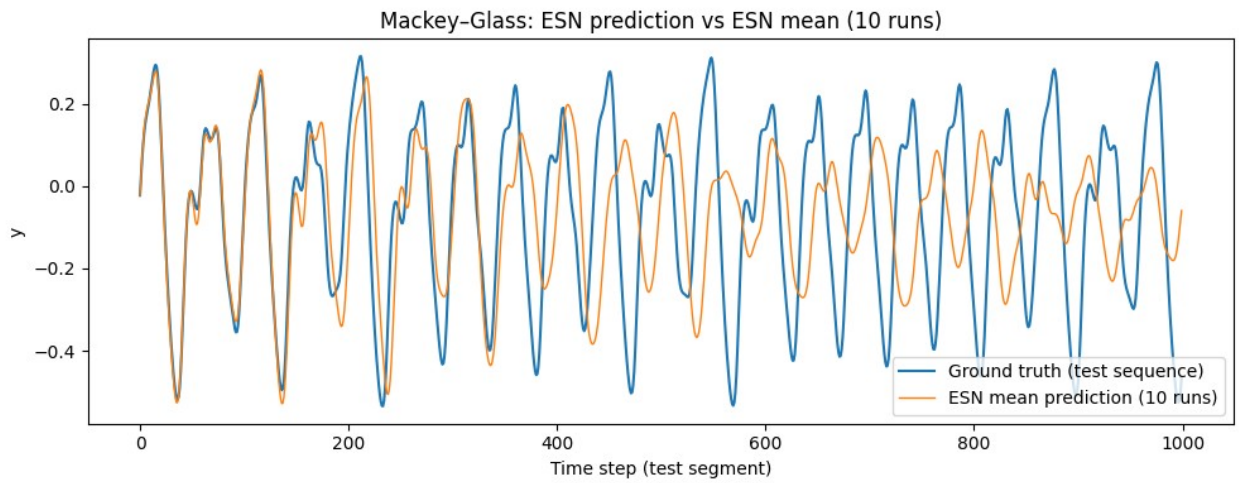
```
# Mackey-Glass series
length = 4000 #Task 1
tau = 17
y = mackey_glass(tau, length)

# Split training
```

```
train_len = 3000 #Task 2
test_len = 1000

train_data = y[:train_len]
test_data  = y[train_len:train_len + test_len]

# Parameters
reservoir_size   = 1000        # Task 3. N_x = 1000 neurons
input_size       = 1
output_size      = 1
spectral_radius  = 0.8         # Task 6. ρ = 0.8
input_scaling    = 0.2         # Task 4. 0.2 Scaling
washout          = 1000        # Task 10. discard first 1000
beta_reg         = 1e-6
n_runs           = 10          # Task 14. repeat for 10 random ESNs

all_predictions = np.zeros((n_runs, test_len))
all_train_rmse  = np.zeros(n_runs)

for run in range(n_runs):
    print(f"Run {run+1}/{n_runs}")

    esn = ESN(
        input_size=input_size,
        reservoir_size=reservoir_size,
        output_size=output_size,
        spectral_radius=spectral_radius,
        input_scaling=input_scaling,
        random_seed=run  # different seed per run
    )

    # Task 7. Inputs during training = ground truth y(n)
    train_inputs = train_data.copy()

    # Task 8. Teaching forcing signal: y(n-1)
    train_inputs = np.zeros_like(train_data)
    train_inputs[1:] = train_data[:-1]   # for n>=1, input is y(n-1)
    train_inputs[0]  = train_data[0]     # first value, doesn't matter
(will be washed out)

# Target at time n: y(n)
    train_targets = train_data.copy()   # first value, washed out


    train_rmse = esn.train(
        inputs=train_inputs,
        targets=train_targets,
        washout=washout,
        beta=beta_reg
    )
```

```
    all_train_rmse[run] = train_rmse
    print(f"  Training RMSE (after washout): {train_rmse:.6f}")

    # Task 11 & 12.
    initial_input = train_data[-1]

    # Task 13: generate predictions for all 1000 test steps
    preds = esn.generate(n_steps=test_len,
initial_input=initial_input)

    all_predictions[run, :] = preds

# Task 14. Plot ground truth vs mean prediction over 10 runs

mean_predictions = all_predictions.mean(axis=0)

plt.figure(figsize=(10, 4))
plt.plot(test_data, label="Ground truth (test sequence)",
linewidth=1.5)
plt.plot(mean_predictions, label="ESN mean prediction (10 runs)",
linewidth=1.0)
plt.xlabel("Time step (test segment)")
plt.ylabel("y")
plt.title("Mackey–Glass: ESN prediction vs ESN mean (10 runs)")
plt.legend()
plt.tight_layout()
plt.show()


from numpy.linalg import norm
test_rmse_per_run = np.sqrt(np.mean((all_predictions - test_data)**2,
axis=1))
print("\nRMSE per run on test sequence:")
print(test_rmse_per_run)
print(f"\nMean test RMSE over 10 runs:
{test_rmse_per_run.mean():.6f}")
```

```
Run 1/10
  Training RMSE (after washout): 0.000215
Run 2/10
  Training RMSE (after washout): 0.000213
Run 3/10
  Training RMSE (after washout): 0.000197
Run 4/10
  Training RMSE (after washout): 0.000228
Run 5/10
  Training RMSE (after washout): 0.000219
Run 6/10
  Training RMSE (after washout): 0.000196
Run 7/10
```

```
  Training RMSE (after washout): 0.000218
Run 8/10
  Training RMSE (after washout): 0.000212
Run 9/10
  Training RMSE (after washout): 0.000215
Run 10/10
  Training RMSE (after washout): 0.000217
```




```
RMSE per run on test sequence:
[0.27996999 0.27839289 0.28449427 0.27745478 0.24980593 0.26811152
 0.29224392 0.28076171 0.1611797  0.29818274]

Mean test RMSE over 10 runs: 0.267060
```

Task 4: Time-series classification with ESNs

```
# Hyperparameters
reservoir_size  = 800        # Task 3. N_x = 800 neurons
input_size      = 1
output_size     = n_classes
spectral_radius = 0.99       # Task 6. ρ = 0.99
input_scaling   = 0.25       # Task 4. Scald by 0.25
beta_reg        = 5e-2       # ridge regularization
n_runs          = 10         # Task 14

accuracies = np.zeros(n_runs)

for run in range(n_runs):
    print(f"\n=== Run {run+1}/{n_runs} ===")


    esn = ESN(
        input_size=input_size,
```

```
        reservoir_size=reservoir_size,
        output_size=output_size,
        spectral_radius=spectral_radius,
        input_scaling=input_scaling,
        random_seed=run
    )

    # Task 7. Encode training signals into last reservoir states

    X_train_states = np.zeros((n_train, reservoir_size))
    for i in range(n_train):
        X_train_states[i, :] = encode_sequence_last_state(esn,
X_train_series[i])

    # Task 8. One-hot encoding of class labels

    label_to_index = {lab: idx for idx, lab in enumerate(classes)}
    y_train_idx = np.array([label_to_index[lab] for lab in
y_train_labels])
    y_test_idx  = np.array([label_to_index[lab] for lab in
y_test_labels])

    Y_train_onehot = np.zeros((n_train, n_classes))
    Y_train_onehot[np.arange(n_train), y_train_idx] = 1.0

    # Task 9. Estimate readout matrix

    X_bias = np.ones((n_train, 1))
    X_aug  = np.hstack([X_bias, X_train_states])     # shape:
(n_train, 1+N_res)
    X_aug_T = X_aug.T                                # (1+N_res,
n_train)

    # Y: (n_classes x n_train)
    Y_T = Y_train_onehot.T

    G = X_aug_T @ X_aug_T.T
    Wout = Y_T @ X_aug_T.T @ np.linalg.inv(G + beta_reg * np.eye(1 +
reservoir_size))

    # store in ESN
    esn.Wout = Wout

    # Task 10,11,12.

    correct = 0
    for i in range(n_test):
        # Task 10: initialize reservoir to zero and feed-in signal
        x_last = encode_sequence_last_state(esn, X_test_series[i])
```

```python
        # Task 11: use last state to make prediction
        x_aug = np.concatenate(([1.0], x_last))
        y_pred_vec = esn.Wout @ x_aug

        # Task 12: predicted class = argmax
        pred_idx = np.argmax(y_pred_vec)

        if pred_idx == y_test_idx[i]:
            correct += 1

    # Task 13. Accuracy
    accuracy = correct / n_test
    accuracies[run] = accuracy
    print(f"Test accuracy (run {run+1}): {accuracy:.4f}")

# Task 15. Mean Accuracy

mean_acc = accuracies.mean()
std_acc  = accuracies.std()
print("\nAccuracies over 10 runs:", accuracies)
print(f"Mean accuracy over 10 runs: {mean_acc:.4f} ± {std_acc:.4f}")
```

```
=== Run 1/10 ===
Test accuracy (run 1): 0.7536

=== Run 2/10 ===
Test accuracy (run 2): 0.7296

=== Run 3/10 ===
Test accuracy (run 3): 0.7488

=== Run 4/10 ===
Test accuracy (run 4): 0.7568

=== Run 5/10 ===
Test accuracy (run 5): 0.7472

=== Run 6/10 ===
Test accuracy (run 6): 0.7376

=== Run 7/10 ===
Test accuracy (run 7): 0.7488

=== Run 8/10 ===
Test accuracy (run 8): 0.7456

=== Run 9/10 ===
Test accuracy (run 9): 0.7376

=== Run 10/10 ===
```

```
Test accuracy (run 10): 0.7424

Accuracies over 10 runs: [0.7536 0.7296 0.7488 0.7568 0.7472 0.7376
0.7488 0.7456 0.7376 0.7424]
Mean accuracy over 10 runs: 0.7448 ± 0.0077
```